

Chapitre supplémentaire de Deep Learning 3, Une brève histoire des réseaux de neurones.

Mai 2026

1 Architecture pour la vision

1.1 Le Net-5

Utilisé sur MNIST (un des plus grands dataset de l'époque), Yann LeCun était intéressé par la lecture automatique de codes postaux sur les lettres. Ce modèle date de 1998.

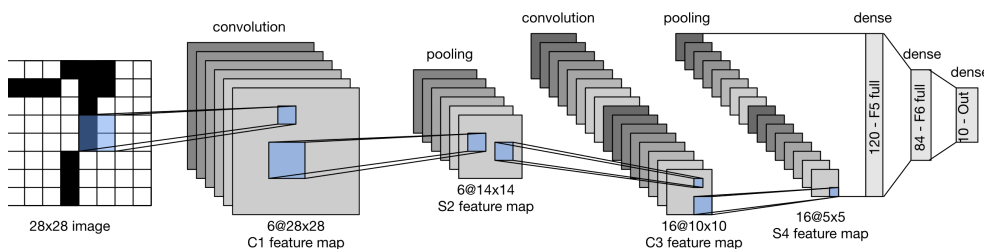


Figure 1: Architecture LeNet-5

Note : fonction d'activation : tanh Utilisation de l'Average Pooling pour réduire progressivement la taille des images. Nombre de paramètres : 60 000 (taille de MNIST). Garder un nombre de paramètres assez petit pour éviter le surapprentissage, ce qui n'est pas le cas aujourd'hui, on sait que l'on peut éviter le surapprentissage avec beaucoup plus de paramètres.

1.2 AlexNet

Compétition ImageNet(3*224*224). Date : 2012. Entre 1998 et 2012, il y a eu l'usage des GPU, c'était un modèle énorme (nb de paramètres 60 Millions). Les innovations sont les suivantes :

- usage de GPUs

- usage de ReLU comme fonction d'activation (pas de saturation comme avec tanh).
- usage de MaxPool, avantage peu sensible aux petites variations de valeurs des pixels =, effet régularisant.
- architecture similaire à LeNet-5 mais avec des couches de convolutions empilées sans MaxPool.

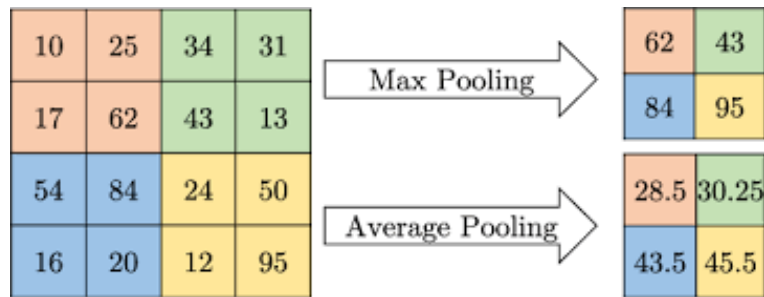


Figure 2: Max Pool

1.3 VGG Nets

L'application est toujours ImageNet en 2014 et le nombre de paramètres est de plus de 100 Millions. Il y a seize couche et non pas cinq. On commence à voir apparaître des blocs. On regroupe les couches en blocs pour pouvoir visualiser les couches regroupées.

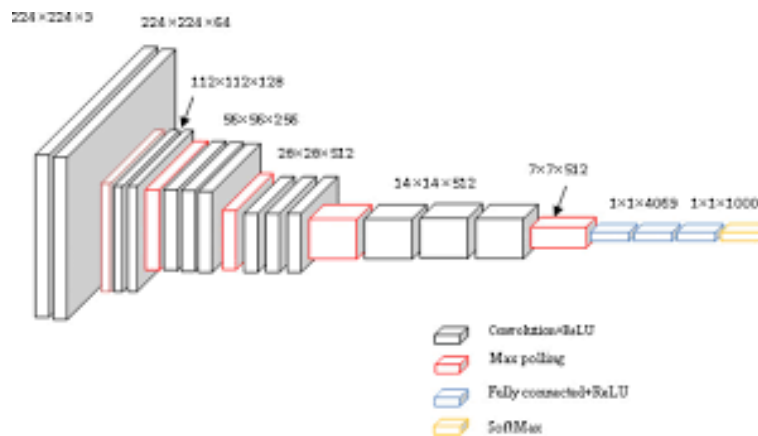


Figure 3: Architecture LeNet-5

La convolution se faisait avec des noyaux 3×3 . Les innovations sont les suivantes :

- Apparition de "blocs" de couches de profondeur variable
- Transition entre chaque bloc : on diminue la taille des images en augmentant le nombre de canaux (ce qui évite de perdre trop d'information)
- utilisation de ReLU entre chaque couche

1.4 GoogleNet/ Inception Networks

Application sur ImageNet, en 2014-2015. Cette fois-ci, le nombre de paramètres est autour de 6 à 7 millions (décroît considérablement). Le modèle précédent était radical car il accumule les couches sans rien faire de réellement intelligent. L'idée ici est de construire des blocs avec plusieurs couches en parallèle.

Convolution 1×1 : presque comme une couche fully-connected, qui résume chaque canal en mettant un poids devant, c'est une manière de réduire la dimension.

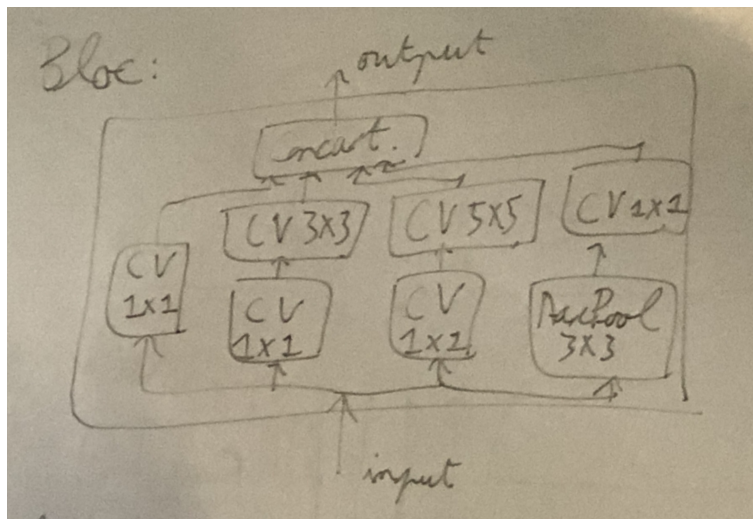


Figure 4: Un bloc

Les innovations sont les suivantes :

- utilisation de plusieurs couches en parallèle de chaque bloc
- CV 1×1 simple combinaison linéaire des canaux $\Rightarrow$ utilisée pour réduire le nombre de canaux.
- version 2016, remplacer une convolution 5×5 par deux convolutions 3×3 . Cela permet de passer de 25 paramètres à 18 paramètres (pour un patch).

Quel est le coût ? Toutes les convolutions 5×5 imaginables ne peuvent pas forcément être représentées par des convolutions 3×3 ...

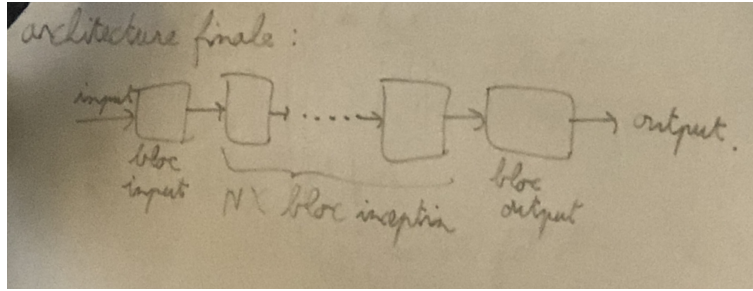


Figure 5: Architecture finale

1.5 ResNet, Residual Networks

Application d'ImageNet en 2015, le nombre de paramètres est très variable, entre 12 et 50 Millions et entre 18 et 152 couches. A cause de la rétropropagation du gradient et la perte de l'information à chaque couche, il y avait des problèmes dans l'entraînement lorsqu'on augmentait les couches du réseau.

L'idée est d'utiliser des connexions résiduelles entre couches, pour garder l'information.

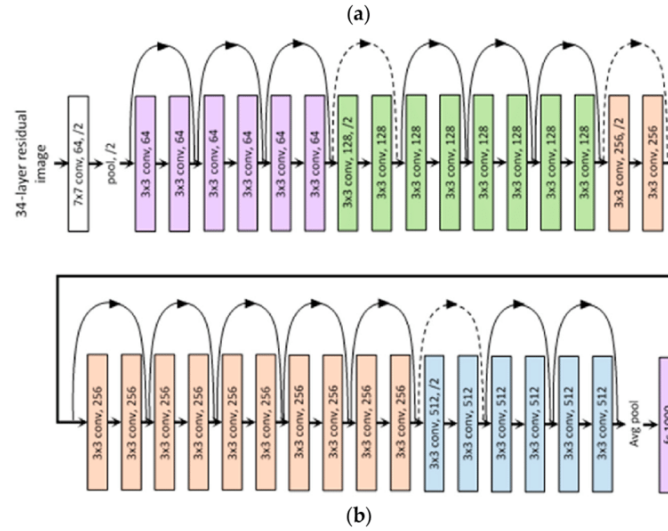


Figure 6: Architecture ResNet

Les residual connexions permettent de s'assurer que l'information sur l'entrée ou la sortie circule (prop et backprop). NB : si on initialise toutes les couches à 0, le réseau interne est initialisé à l'identité. Auquel cas, on apprend une déformation des données.

Note : On est contraint par rapport au nombre de canaux, à l'intérieur de

chaque bloc, si on met des residual connexions, il faut garder le même nombre de canaux. Le fait de préserver de manière certaine l'info du début à la fin fait que les réseaux types resNet ont été plus étudiés en terme de complexité (plus facile à étudier). Plus les réseaux de neurones sont profonds, plus ils sont expressifs. On peut utiliser du batchNorm et du layerNorm pour faire en sorte que l'information soit bien communiquée. Mais cela a répondu au fléau des vanishing gradients.

1.6 U-Nets

Application pour de la segmentation d'images (grandes en générale) médicales, en 2015, le nombre de paramètres est de 250 Millions.

NB : Utilisés aussi pour de la génération d'images dans les modèles de diffusion (pour débruiter les images).

L'idée est de partir d'une image en input, faire traverser un certain nombre de couches convolutionnelles (généralement 3), et à la sortie, parcours de deux circuits (dawnsampling, typiquement Maxpool) et on lui fait arriver dans les couches de sortie (4 dont la dernière est fully-connected). L'image va être traitée de manière multi-échelle. On va d'abord utiliser des patches 3x3 pour regarder les détails puis on regarde ce qu'il se passe à échelle plus large et on recommence le traitement de couches convolutionnelles, avec des couches à la fin (lobbsampling = transpose convolution) pour obtenir des images de la même taille. Et on répète ça un certain nombre de fois (originellement 4 fois) etc...

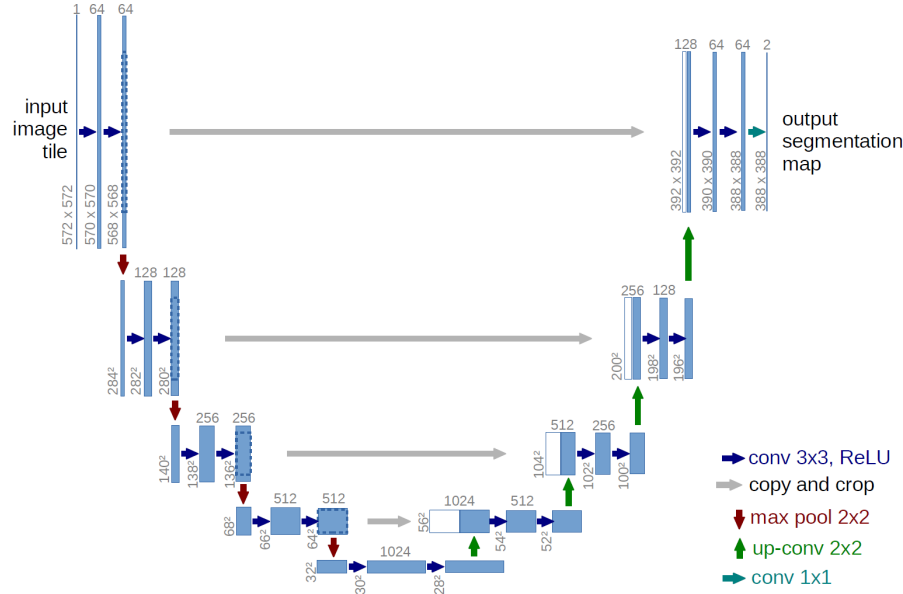


Figure 7: Architecture U-Net

Dans la pratique, et c'est le cas de tous les réseaux présentés ci-dessus, le code ne reflète pas exactement ce schéma.

2 Compression de modèles

ENJEUX :

- Problème du choix de l'architecture (nombre de neurones, nombre de couches ou même structures des blocs)
- Coût d'entraînement et d'utilisation des modèles.

2.1 Méthode d'élagage fondée sur la hessienne

Optimal Brain Damager (1989, LeCun). L'idée est, à la fin, de l'entraînement, on détecte les paramètres qui influent peu sur le résultat, afin de les mettre à 0 (i.e. les supprimer).

$$L(\theta + h) = L(\theta) + h^T \text{Gradient}(L(\theta)) + \lambda/2 * h^T \text{Gradient}^2(L(\theta)) + o(||h||^2) \quad (1)$$

En supposant que l'on est à la fin de l'entraînement, on a que le gradient de la loss doit être égal à zéro.

Rappel : on veut élaguer, donc on soustrait $\theta_p * e_p$ à θ , ou e_p est le vecteur élémentaire. Donc on effectue la procédure sur:

Défauts :

- Approximation irréalisable car le gradient n'est jamais nul
- C'est de l'élagage non structuré (pas de mise à 0 de couches entières et de réseaux entiers).

Développement : Optimal Brain Surgeon (Harsabi, 1993)

2.2 Elagage structuré

2.2.1 Group-Lasso

Rappel: pénalité Lasso, norme 1, induit de la sparsité.

Donc le problème : la pénalité L2 n'induit pas de sparsité. La pénalité Lasso induit de la sparsité mais non structurée, donc l'idée est de combiner les deux. On considère une partition des paramètres du réseau. $\theta = (T_1, \dots, T_s)$. La pénalité group Lasso est :

$p(T_1, \dots, T_s) = \sum [i = 1][s] \sqrt{p_s} * ||T_s||_2$. Où p_s est le nombre de paramètres dans T_s .

Intuition : on applique deux normes différentes aux différentes échelles, individuellement, un paramètre n'est pas censé atteindre 0 (mais collectivement)

soit $\mathcal{L}(\theta)$ la fonction

On a :

$$\begin{aligned}\mathcal{L}(\theta+h) &= \mathcal{L}(\theta) + h^T \nabla \mathcal{L}(\theta) \\ &\quad + \frac{1}{2} h^T \nabla^2 \mathcal{L}(\theta) h \\ &\quad + o(\|h\|^2)\end{aligned}$$

En supposant qu'on est à la fin de l'entraînement,
on a : $\nabla \mathcal{L}(\theta) = 0$

$$\begin{aligned}\text{donc : } \mathcal{L}(\theta+h) &\approx \mathcal{L}(\theta) + \frac{1}{2} h^T \nabla^2 \mathcal{L}(\theta) h \\ &\approx \mathcal{L}(\theta) + \frac{1}{2} \sum_{r=1}^p h_r \frac{\partial^2 \mathcal{L}}{\partial \theta_r \partial \theta_r} h_r\end{aligned}$$

Hyp. simplificatrice : si $p \neq q$, alors $\frac{\partial^2 \mathcal{L}}{\partial \theta_r \partial \theta_q} = 0$

$$\text{Donc : } \mathcal{L}(\theta+h) \approx \mathcal{L}(\theta) + \frac{1}{2} \sum_{r=1}^p h_r^2 \frac{\partial^2 \mathcal{L}}{\partial \theta_r^2}$$

Donc on effectue la procédure suivante :

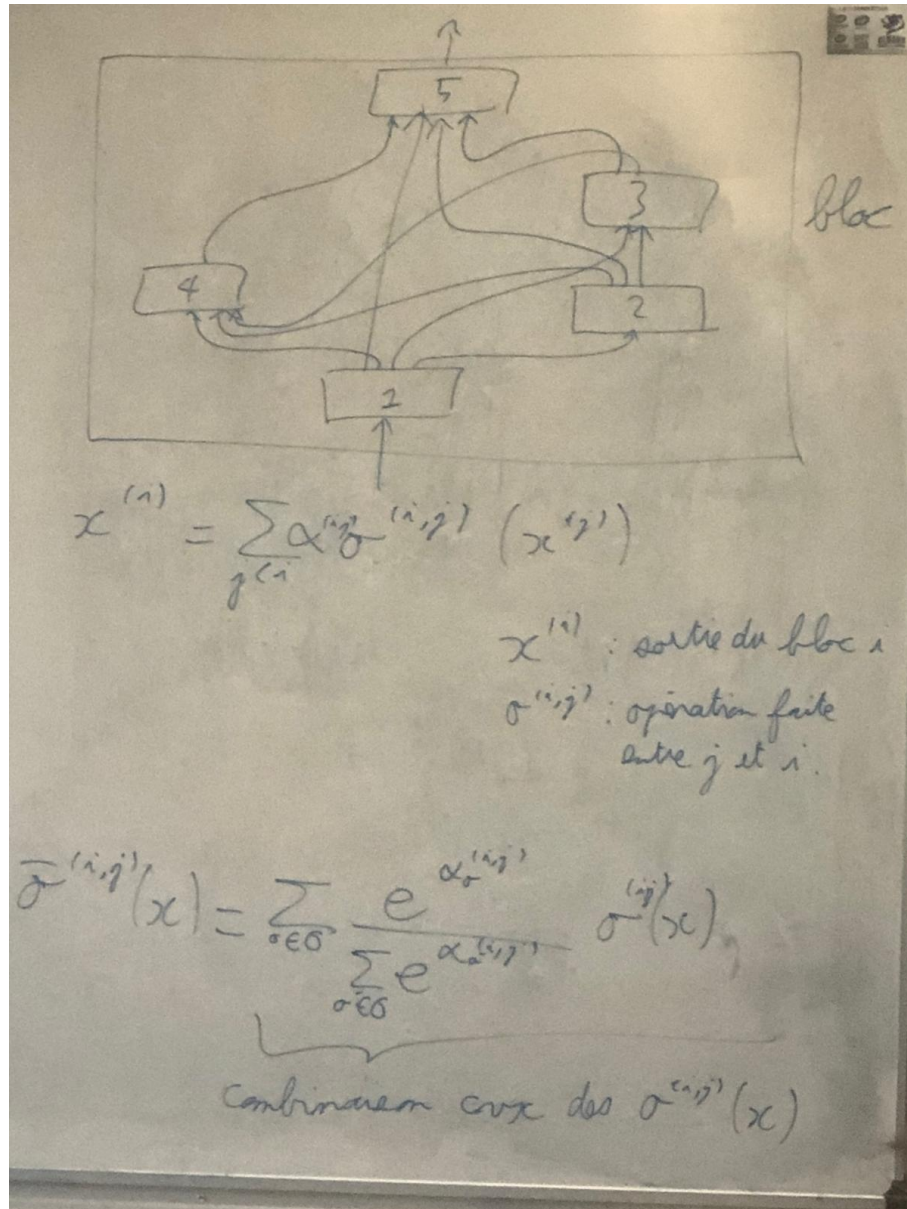
Pour p de 1 à P :

$$\text{on calcule } \delta_p := \theta_r^i \frac{\partial^2 \mathcal{L}}{\partial \theta_r^2}$$

On sélectionne $p^* = \operatorname{argmin}(\delta_p)$

2.3 DARTS : Differentiable Architecture Search

Lin, 2018. Le but est d'entraîner une architecture de réseau, de blocs, pour une réseau de neurones.



Cela permet de récupérer seulement les flèches significatives et mettre les paramètres à 0.

Autre chose : encoder les paramètres sur moins d'octets (2 à 3). Ou encore remplacer les matrices de poids par des matrices de faible rang (stocker des petites matrices).